
JAVASCRIPT RECAP

◆ 1. Variables (`var` , `let` , `const`) – Summary

JavaScript offers three ways to declare variables:

- `var` – function-scoped, hoisted, and can be re-declared.
- `let` – block-scoped, not hoisted in the same way as `var`.
- `const` – block-scoped, must be initialized at declaration, and can't be reassigned (but objects can still be mutated).

Common Pitfalls:

- `var` leaks outside blocks like `if` or `for`.
 - `const` prevents reassignment, not mutation.
-

◆ 2. Data Types – Summary

JavaScript has two categories of data types:

Primitive Types (immutable):

- `String`
- `Number`
- `Boolean`
- `null`
- `undefined`
- `Symbol`
- `BigInt`

Non-Primitive Types (mutable, reference types):

- `Object` (includes arrays, functions, etc.)

Important Notes:

- `typeof null` is `"object"` (a known JS quirk).
- Primitives are compared by **value**, objects by **reference**.

◆ 3. Hoisting – Summary

Hoisting is JavaScript's behavior of moving **declarations** (not initializations) to the top of their scope during compilation.

- `var` declarations are **hoisted** with `undefined` as initial value.
- `let` and `const` are hoisted too, but in a "**temporal dead zone**"—accessing them before declaration throws a `ReferenceError`.
- Functions declared using `function` are fully hoisted, while function expressions are not.

◆ 4. Closures – Summary

A **closure** is when a function "remembers" its **lexical scope** even after the outer function has finished executing.

```
function outer() {  
  let counter = 0;  
  return function inner() {  
    return ++counter;  
  };  
}  
  
const count = outer();  
count(); // 1  
count(); // 2
```

Real-World Uses:

- Data encapsulation
- Maintaining state (e.g., counters)
- Currying and partial applications

✅ Interview Q&A (Role: Candidate)

◆ 1. What's the difference between `var`, `let`, and `const`?

Answer: `var` is function-scoped and can be re-declared. `let` and `const` are block-scoped. `const` also prevents reassignment. I prefer `let` for variables that change and `const` for values that stay constant to improve code safety and clarity.

◆ 2. What is the output of this code?

```
console.log(a);  
var a = 5;
```

Answer: It logs `undefined`. Due to hoisting, `var a` is moved to the top and initialized as `undefined`, then assigned `5`.

◆ 3. What is a closure, and how do you use it?

Answer: A closure is when a function retains access to its lexical scope. I use closures to keep state between function calls. For example, a counter function can remember how many times it's been called using closure.

◆ 4. What are primitive vs non-primitive data types?

Answer: Primitive types (like `number`, `string`, `boolean`) hold value directly. Non-primitive types (like objects and arrays) hold references. Comparing objects compares references, not structure or content.

◆ 5. Why does `typeof null` return `"object"`?

Answer: It's a legacy bug in JavaScript. Internally, `null` was represented as a null pointer, which was considered an object. It's recognized but not fixed for backward compatibility.

◆ 6. What is the Temporal Dead Zone?

Answer: The time between hoisting a `let` / `const` variable and its actual declaration is the **Temporal Dead Zone**. Accessing it before it's declared results in a `ReferenceError`.

◆ 7. Can you give an example of a function that uses a closure?

Answer: Sure:

```
function makeMultiplier(x) {  
  return function(y) {  
    return x * y;  
  };  
}
```

```
const double = makeMultiplier(2);  
console.log(double(5)); // 10
```

Here, `double` keeps a reference to `x = 2` due to closure.

◆ 5. Async/Await – Summary

`async` and `await` are syntax sugar over Promises, making asynchronous code look synchronous.

- `async` marks a function that always returns a Promise.
- `await` pauses execution inside `async` until the Promise settles (fulfilled or rejected).

```
async function fetchData() {  
  const res = await fetch("https://api.example.com/data");  
  const data = await res.json();  
  return data;  
}
```

Key Points:

- Use `try...catch` for error handling.
 - `await` only works inside `async` functions.
 - Allows sequential and parallel async calls.
-

◆ 6. Prototypes – Summary

Every JavaScript object has a **prototype** from which it can inherit properties and methods.

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.greet = function() {  
  return `Hello, ${this.name}`;  
};  
const p = new Person("Alice");  
p.greet(); // "Hello, Alice"
```

Notes:

- `__proto__` is the object's reference to its prototype.

- Prototype chain is the basis of JavaScript inheritance.

◆ 7. Scope – Summary

Scope defines where variables can be accessed.

Types of scope:

- **Global Scope** – accessible anywhere
- **Function Scope** – `var`
- **Block Scope** – `let` , `const`

```
function test() {  
  if (true) {  
    let a = 5; // block-scoped  
  }  
  console.log(a); // ReferenceError  
}
```

Closely related to:

- Lexical scope (defined by code structure)
- Shadowing (inner variables overriding outer ones)

◆ 8. Event Delegation – Summary

Event Delegation allows you to handle events at a **parent level**, leveraging **event bubbling**.

```
document.getElementById("list").addEventListener("click", (e) => {  
  if (e.target.tagName === "LI") {  
    console.log("Clicked:", e.target.textContent);  
  }  
});
```

Use Cases:

- Dynamic content
 - Fewer event listeners
 - Performance optimization
-



Interview Q&A (Role: Candidate)

◆ 1. What's the benefit of `async/await` over Promises?

Answer: `async/await` simplifies the syntax, making async code easier to read and maintain. Instead of chaining `.then()` and `.catch()`, I can write logic in a top-down manner and use `try/catch` for errors.

◆ 2. What happens if I `await` a non-promise value?

Answer: It wraps the value in a resolved Promise. So `await 5` is like `Promise.resolve(5)`.

◆ 3. What is a prototype and how is it used in JavaScript?

Answer: A prototype is an object that other objects inherit from. For instance, functions like `Array.prototype.map` are shared by all arrays via the prototype chain. I use it to define shared methods across instances.

◆ 4. What's the difference between scope and closure?

Answer: Scope is the current context of execution, determining variable access. A closure is a function that retains access to its outer scope even after that scope is gone. Closures are formed because of lexical scope.

◆ 5. What is event delegation and why is it useful?

Answer: Event delegation uses a single handler at a parent element to manage events from its children. It improves performance and handles dynamically added elements. I often use it in lists or tables.

◆ 6. What's the difference between `__proto__` and `prototype`?

Answer: `__proto__` is an object's actual prototype reference. `prototype` is a property on constructor functions used to assign shared methods or properties to instances created by `new`.

◆ 7. Can you show an example where event delegation is better than multiple listeners?

Answer: Sure:

```
<ul id="menu">
  <li>Home</li>
  <li>About</li>
</ul>

<script>
document.getElementById("menu").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    alert(`You clicked ${e.target.textContent}`);
  }
});
</script>
```

This single listener handles all `<li>` items, including ones added dynamically.

◆ 9. Event Loop – Summary

The **Event Loop** is a core part of JavaScript's concurrency model. It allows non-blocking I/O by offloading operations and scheduling callbacks via a **task queue**.

Flow:

1. JS runs in a **single-threaded call stack**.
2. Long tasks like HTTP requests or `setTimeout()` go to the **Web APIs**.
3. When done, callbacks go to the **callback queue**.
4. The **Event Loop** pushes them back to the call stack when it's empty.

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
console.log("End");
// Output: Start → End → Timeout
```

◆ 10. Functional Programming – Summary

JavaScript supports **functional programming** principles such as:

- First-class functions (functions are values)
- Pure functions
- Higher-order functions
- Immutability
- Recursion

Functional methods:

- `.map()`, `.filter()`, `.reduce()`, `.forEach()`

```
const nums = [1, 2, 3];  
const doubled = nums.map(n => n * 2); // [2, 4, 6]
```

◆ 11. Callbacks – Summary

A **callback** is a function passed into another function to be **executed later**.

```
function greet(name, cb) {  
  cb(`Hello, ${name}`);  
}  
greet("Sam", console.log); // Hello, Sam
```

Callback Hell:

When callbacks are deeply nested, causing unreadable code.

```
doTask1(() => {  
  doTask2(() => {  
    doTask3(() => {  
      // ...hell!  
    });  
  });  
});
```

◆ 12. Destructuring – Summary

Destructuring allows **unpacking values** from arrays or objects into separate variables.

```
const [a, b] = [1, 2]; // a=1, b=2
```



```
const { name, age } = { name: "John", age: 30 };
```

Advanced:

- Nested destructuring
- Default values
- Destructuring function parameters

```
function display({ name = "Guest" }) {  
  console.log(name);  
}
```

✓ Interview Q&A (Role: Candidate)

◆ 1. Can you explain the Event Loop in JavaScript?

Answer: The Event Loop handles async behavior. While JS is single-threaded, it offloads async tasks to Web APIs. Once those tasks complete, their callbacks are queued. The event loop ensures these callbacks run only when the call stack is empty, maintaining non-blocking execution.

◆ 2. What is functional programming and how does JavaScript support it?

Answer: Functional programming emphasizes immutability and pure functions. JavaScript supports this with first-class functions, and methods like `.map()`, `.filter()`, and `.reduce()` that allow declarative, side-effect-free transformations.

◆ 3. How do you avoid callback hell?

Answer: I use Promises or `async/await` to flatten callback chains. They provide better readability and error handling. Libraries like `async.js` also help, but ES6 features made native solutions easier.

◆ 4. What is the output?

```
console.log("1");  
setTimeout(() => console.log("2"), 0);
```

```
Promise.resolve().then(() => console.log("3"));  
console.log("4");
```

Answer:

```
1  
4  
3  
2
```

Explanation: `Promise.then` is a **microtask** and executes before `setTimeout`, which is a **macrotask**.

◆ 5. What's the benefit of destructuring?

Answer: It improves readability and reduces code size when working with objects or arrays. Especially helpful when handling function arguments or working with structured API responses.

◆ 6. Can you show object destructuring in a function?

Answer: Sure:

```
function greet({ name, age }) {  
  console.log(`Hello ${name}, you are ${age}`);  
}  
greet({ name: "Alice", age: 25 });
```

◆ 7. What is a higher-order function?

Answer: A function that takes another function as an argument or returns a function. Common in functional programming. Example: `.map()` takes a function and applies it to each element.

◆ 17. Memory Management – Summary

JavaScript uses **automatic garbage collection** to manage memory. It allocates memory when variables/objects are created and frees it when they are no longer reachable.

Key Concepts:

- **Heap:** Memory pool for objects/functions
- **Stack:** For call frames and primitives
- **Garbage Collector:** Removes unreachable memory (e.g., orphaned objects)

Memory Leaks:

- Global variables
- Closures holding references
- Detached DOM elements
- Timers (`setInterval`) not cleared

◆ 18. Debugging – Summary

Debugging is the process of finding and fixing bugs in your code.

Tools & Techniques:

- `console.log()` , `console.error()` , `console.table()`
- Browser DevTools (`Sources` , `Network` , `Elements` , etc.)
- Breakpoints and step execution
- Watch expressions
- Stack traces

Use `debugger;` to pause code at runtime.

◆ 19. Testing – Summary

Testing ensures code behaves as expected. JavaScript supports:

Types:

- **Unit Tests** – test individual functions
- **Integration Tests** – test interactions between components
- **E2E Tests** – test user flows (e.g., Cypress, Playwright)

Tools:

- Jest (unit testing)
- Mocha + Chai
- Testing Library (React, DOM)

- Cypress (E2E)
-

◆ 20. Dev Tools – Summary

Browser DevTools provide deep insight into how your code runs.

Key Tabs:

- **Elements** – Inspect/edit DOM
 - **Console** – Log and test snippets
 - **Sources** – Debug JS
 - **Network** – Monitor API requests
 - **Performance** – Analyze runtime performance
 - **Memory** – Find leaks
 - **Application** – View cookies, storage
-



Interview Q&A (Role: Candidate)

◆ 1. How does JavaScript manage memory?

Answer: JS manages memory automatically with a garbage collector. It frees memory when objects are no longer reachable, such as when references are dropped. I avoid leaks by cleaning up timers, listeners, and unused variables.

◆ 2. What are some causes of memory leaks in JS?

Answer: Common causes include:

- Forgotten `setInterval` or event listeners
 - Closures retaining references
 - Detached DOM elements still referenced
 - Large global variables
-

◆ 3. How do you debug a JavaScript application?

Answer: I use browser DevTools to set breakpoints, watch variables, and step through execution. I also log state with `console.log()` and inspect the call stack when errors occur. For complex issues, I use the `debugger` statement to pause execution manually.

◆ 4. What is the difference between unit, integration, and E2E testing?

Answer:

- **Unit:** Tests single functions in isolation (e.g., Jest)
- **Integration:** Tests combined modules or services
- **E2E:** Simulates full user interactions (e.g., with Cypress or Playwright)

Each layer ensures a different level of confidence.

◆ 5. What tools do you use for testing?

Answer:

- **Jest:** For unit tests
 - **React Testing Library:** For component testing
 - **Cypress:** For E2E tests These help catch regressions, validate UI logic, and ensure reliability before release.
-

◆ 6. What DevTools features do you use most often?

Answer:

- **Network** to monitor API calls
 - **Console** for logs
 - **Sources** for debugging and breakpoints
 - **Performance** to check frame drops or slow rendering
 - **Application** to inspect localStorage/sessionStorage and cookies
-

◆ 7. How do you check for performance bottlenecks in JS?

Answer: I use the **Performance tab** in DevTools to record sessions and check CPU usage, layout shifts, and JS execution time. I also use `console.time()` and `console.profile()` to manually benchmark blocks of code.
